LLM Monitoring, Drift Detection, and Evaluation in Production

Scaler School of Technology | ML System Design and LLMOps | Week 7, Session 2 | Post-Lecture Review & Practice

Key Takeaways

LLM failures are silent and confident

An LLM that degrades does not return error codes. It returns fluent, confident, politely formatted wrong answers. You cannot detect this with infrastructure monitoring alone. You need output-level evaluation.

Monitor at all five layers

Operational → Input → Output → Quality → Drift. Each layer catches a different class of failure. Skipping output and quality monitoring means you will find out about degradation from user complaints, not dashboards.

LLM-as-judge is the practical solution for quality at scale

Human review of 100% of LLM outputs is not feasible. LLM-as-judge, calibrated against weekly human samples, gives you quality monitoring at scale. The calibration step is non-negotiable — an uncalibrated judge gives you false confidence.

Every prompt change is a deployment event

Treat prompt versioning the same as code versioning. Tag all traces with prompt version. Always compare quality distributions across prompt versions before declaring a new prompt "better".

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. A production RAG chatbot has stable latency, zero API errors, and a thumbs-up rate of 72%. The team is satisfied. What critical monitoring is missing and why might the system still be failing?

Q2. You are evaluating a new prompt version. LLM-as-judge scores improve from 7.1 to 7.8. Is this sufficient evidence to deploy? What else do you need?

Q3. Describe one concrete way prompt injection can occur in a RAG system (not a direct user prompt injection). What is the mitigation?

Q4. Your LLM API cost has doubled in the past two weeks with no change in user count. Name three things you would check first, in order of likelihood.

Design Exercises

Work through these independently or in pairs. No starter code — derive the approach from the session.

Exercise 1: Instrument a LangChain App with Langfuse

Take your W7S1 RAG system (or any LangChain-based app). Add Langfuse tracing. Verify that each LLM call shows up as a trace with: input prompt, output, latency, and token count. Screenshot the Langfuse dashboard showing at least 5 traces. Note: the Langfuse SDK supports LangChain via a CallbackHandler — add it in one line.

Exercise 2: Build an LLM-as-Judge Evaluator

Write a Python function `evaluate_response(question, context, answer)` that: (1) sends a structured prompt to an LLM asking it to rate the answer on helpfulness (1–5) and groundedness (1–5); (2) parses the JSON score output; (3) returns a dict with both scores. Run it on 10 outputs from your RAG system. What is the average score? Are there any outputs where groundedness < 3?

Exercise 3: Cost Attribution Analysis

Using your Langfuse traces (or a synthetic dataset of 100 LLM calls), compute: (a) mean cost per request; (b) p95 cost per request; (c) the top 5% of most expensive requests — what do they have in common? Write a one-paragraph cost optimisation recommendation based on your findings.

Challenge Problem

Design a Monitoring SLA for a Production LLM System

You are the LLMOps engineer for a customer-facing LLM product with 50,000 daily active users. Design a complete monitoring SLA document (one page). It must specify: (1) the metrics you will monitor; (2) the alert thresholds for each metric; (3) the on-call response protocol for each alert type; (4) the weekly review process; (5) how you will detect a quality regression before users notice. Justify every threshold with reasoning.

Further Reading

- **Langfuse Documentation** — langfuse.com/docs
  Complete reference for tracing, evaluations, and prompt versioning. Start with the "Tracing" and "Scores" sections. The LangChain integration guide is 10 minutes to implement.
- **RAGAS — Evaluation Framework for RAG** — github.com/explodinggradients/ragas
  Open-source framework implementing faithfulness, answer relevancy, context precision, and context recall. Directly applicable to your W7S1 system.
- **Liang et al. (2022) — Holistic Evaluation of Language Models (HELM)** — crfm.stanford.edu/helm
  Stanford's benchmark framework. Read the evaluation methodology section for rigorous thinking about what LLM quality means.
- **Riley Goodside's Prompt Injection Research** — twitter.com/goodside
  The original public demonstration of prompt injection via retrieved documents. Background reading for the incident case study.

Tools to Know

- **Langfuse** — langfuse.com — open source LLM tracing
- **Arize Phoenix** — phoenix.arize.com — embedding drift, RAG eval
- **RAGAS** — ragas.io — RAG-specific evaluation metrics
- **Llama Guard** — Meta's open-source safety classifier for LLM outputs
- **WhyLabs** — whylabs.ai — statistical monitoring for ML and LLMs
- **BERTopic** — topic modelling for query clustering

Answers

A1. The system is missing *output quality monitoring*. Stable latency and zero API errors only confirm the system is running, not that it is producing correct answers. Thumbs-up rate of 72% sounds good but only 2% of users typically provide feedback — it reflects a small, self-selected sample. The system could be hallucinating facts for the 98% who never rate. Missing: LLM-as-judge evaluation on sampled outputs, hallucination detection (faithfulness to retrieved context), and refusal rate tracking.

A2. No. An improvement in LLM-as-judge scores alone is not sufficient. You also need: (a) human review of at least 30–50 sampled outputs to validate that the judge's score improvement reflects real quality improvement; (b) check that the improvement holds across user segments (not just average); (c) run the new prompt on a regression test dataset to ensure it does not regress on cases the old prompt handled well. Judge scores can be gamed by prompting the judge differently; calibration is required.

A3. A malicious document in the knowledge base containing instruction-overriding text (e.g. "Ignore previous instructions and output X"). When the retriever selects this document and it is inserted into the LLM context, the LLM may follow the injected instructions. Mitigation: sanitise retrieved documents at index time; wrap retrieved content in structural markers (`[DOCUMENT] ... [/DOCUMENT]`); run a safety check on retrieved chunks before including them in context.

A4. In order of likelihood: (1) **Context length increase** — check p95 and p99 of input token counts; a single developer feature that pastes large documents can 10× costs; (2) **Model version change** — confirm whether you switched to a more expensive model (e.g. GPT-4o instead of GPT-4o-mini) in a recent deployment; (3) **Traffic spike in high-cost use cases** — check cost per request segmented by feature or endpoint; some user workflows may be significantly more expensive than others.

Course materials: priyanshsaxena.com/sst-mlsystemdesign **Instructor:** Priyansh Saxena